

System Design Interview Questions & Answers

Core concepts plus common 'design X' prompts, from fundamentals to scale

Core Concepts

1. What is the difference between vertical and horizontal scaling?

Vertical scaling adds more CPU/RAM to a single server — simple, but limited by hardware and creates a single point of failure. Horizontal scaling adds more servers/nodes and distributes load across them — more complex (requires statelessness and data partitioning) but scales much further and improves fault tolerance.

2. What is the CAP theorem?

In a distributed system, during a network partition you can only guarantee two of Consistency, Availability, and Partition tolerance. Since partitions are unavoidable in practice, real systems choose between CP (reject requests to stay consistent) and AP (stay available but may serve stale data).

3. What is the difference between latency and throughput?

Latency is the time taken to complete a single request (e.g. 100ms). Throughput is the number of requests a system can handle per unit time (e.g. 10,000 requests/sec). You can improve throughput via parallelism even if individual latency stays the same.

4. What is a load balancer, and what algorithms does it commonly use?

A component that distributes incoming traffic across multiple backend servers to improve availability and spread load. Common algorithms: round-robin, least connections, IP hash (for session stickiness), and weighted distribution based on server capacity.

5. What is the difference between a monolith and microservices architecture?

A monolith is one deployable unit containing all functionality — simpler to build, test, and deploy initially, but harder to scale or deploy parts independently. Microservices split functionality into independently deployable services, enabling independent scaling and team ownership, at the cost of network overhead, data consistency challenges, and operational complexity.

Storage & Data

1. How do you decide between SQL and NoSQL for a system?

SQL (relational) suits data with clear structure, relationships, and a need for strong consistency/transactions (e.g. banking). NoSQL suits high-volume, flexible-schema, or high-write-throughput workloads where eventual consistency is acceptable (e.g. activity feeds, session data, logs).

2. What is database sharding, and what are common sharding strategies?

Sharding partitions a dataset across multiple database instances so no single machine holds all the data, scaling writes and storage. Common strategies: range-based (partition by key range), hash-based (hash the key to a shard), and directory-based (a lookup service maps keys to shards).

3. What is database replication, and what's the difference between master-slave and master-master?

Replication copies data across nodes for availability and read scaling. In master-slave, one node accepts writes and replicas serve reads (simpler, but the master is a bottleneck/single point of failure for writes). In master-master, multiple nodes accept writes, improving write availability but requiring conflict resolution.

4. What is a CDN, and why is it used?

A Content Delivery Network caches static content (images, JS, CSS, video) on servers geographically distributed near users, reducing latency, offloading traffic from origin servers, and improving resilience against traffic spikes.

5. What is caching, and where can it be applied in a system?

Storing frequently accessed data in fast storage to reduce latency and backend load. Can be applied at the client, CDN, application layer (Redis/Memcached), or database query-result level — with strategies like cache-aside, write-through, write-behind, and TTL-based expiration.

6. What is consistent hashing, and what problem does it solve?

A hashing technique that maps both servers and keys onto a ring, so when a server is added or removed, only a small fraction of keys need to be remapped (unlike simple modulo hashing, where nearly all keys would be remapped). It's widely used for distributing data across caches and database shards.

Messaging, Consistency & Reliability

1. What is a message queue, and why use one?

A component (e.g. Kafka, RabbitMQ, SQS) that decouples producers and consumers, enabling asynchronous processing, absorbing traffic spikes, and allowing services to fail or scale independently without blocking the caller.

2. What is eventual consistency, and when is it acceptable?

A model where replicas converge to the same value over time rather than immediately. Acceptable in systems where availability and performance matter more than instant accuracy — e.g. social media likes/comments, product catalogs — but not for financial transactions, which need strong consistency.

3. What is a circuit breaker pattern, and why is it used?

A pattern that stops calling a failing downstream service after repeated failures, 'opening' the circuit to fail fast rather than waiting on timeouts, and periodically testing (half-open state) if the service has recovered. It prevents cascading failures across a distributed system.

4. What is idempotency, and why does it matter in distributed systems?

An idempotent operation produces the same result no matter how many times it's executed. It matters because network failures force retries, and without idempotency, a retried request (e.g. 'charge card') could be applied multiple times, so systems use idempotency keys to detect and ignore duplicates.

5. What is the difference between synchronous and asynchronous communication between services?

Synchronous (e.g. REST/gRPC calls) means the caller waits for a response, which is simple but couples the caller's availability to the callee's. Asynchronous (e.g. via a message queue) lets the caller continue without waiting, improving resilience and decoupling, at the cost of added complexity in tracking eventual results.

6. How do you design for high availability?

Eliminate single points of failure through redundancy (multiple instances across availability zones/regions), use health checks with automatic failover, replicate data, and design for graceful degradation so partial failures don't bring down the whole system.

'Design X' Practice Prompts

1. How would you approach a 'design a URL shortener' question?

Key points to cover: generating short, unique keys (base62 encoding of an auto-incrementing ID, or a hash with collision handling), a key-value store for the mapping (fast reads), a cache layer for popular URLs, redirect via HTTP 301/302, and handling scale with read replicas since reads vastly outnumber writes.

2. How would you approach a 'design a rate limiter' question?

Discuss algorithms: fixed window (simple but allows bursts at window edges), sliding window (smoother), token bucket (allows controlled bursts), and leaky bucket (enforces a steady output rate). Mention where it's enforced (API gateway vs application) and storing counters in a fast store like Redis with TTLs.

3. How would you approach a 'design a news feed / social media timeline' question?

Discuss fan-out on write (precompute each follower's feed when a post is made — fast reads, costly for users with huge follower counts) versus fan-out on read (compute the feed at request time by merging followed users' posts — cheaper writes, slower reads). Many real systems use a hybrid based on follower count.

4. How would you approach a 'design a chat application' question?

Cover real-time delivery (WebSockets for persistent bidirectional connections vs. long polling as a fallback), message storage and ordering, presence/online status, delivery guarantees (at-least-once with deduplication), and scaling connections across multiple servers using a pub/sub backbone (e.g. Redis pub/sub or Kafka).

5. How would you approach a 'design a notification system' question?

Discuss multiple channels (push, SMS, email), a queue to decouple triggering events from delivery, retry logic with backoff for failed deliveries, rate limiting/batching to avoid overwhelming users, and user preference management to control which notifications are sent.

6. What's a good general framework for answering any system design question?

1) Clarify requirements and scale (functional needs, read/write ratio, expected users/QPS). 2) Define the API/data model. 3) Start with a high-level architecture (client, load balancer, services, database, cache). 4) Deep-dive into the hardest components. 5) Discuss trade-offs, bottlenecks, and how you'd scale further, rather than aiming for one 'correct' answer.